

Chord DHT 身份验证扩展设计文档

基于 CA 签发证书 + Ed25519 请求签名的节点身份验证方案

版本 v2.0 | 2026 年 5 月 28 日

文档版本	v2.0
创建日期	2026 年 5 月 28 日
基于版本	v1.0 (Chord DHT 学习系统设计文档)
文档状态	草案
适用对象	在 v1.0 基础上添加节点身份验证功能的工程师
适用范围	学习用途，不适合直接用于生产环境
新增协议栈	Ed25519 (crypto/ed25519)、自定义轻量证书 (JSON)、HTTP 请求签名

目 录

- 第一章 方案概述与可行性结论
- 第二章 身份模型与密钥体系
- 第三章 CA 操作手册
- 第四章 请求签名协议
- 第五章 NodeInfo Schema 变更
- 第六章 证书生命周期管理
- 第七章 Node API 变更规范
- 第八章 Tracker API 变更规范
- 第九章 边缘情况与攻击面分析 (26 个场景)
- 第十章 新增配置参数
- 第十一章 消息时序变更
- 第十二章 实现建议 (Go 语言)

方案概述与可行性结论

1.1 需求描述

v1.0 明确将"TLS / 认证 / 鉴权"列为不在设计范围内 (§1.3)，并指出"节点 ID 由本地 SHA-1 计算，不经过中心化验证" (Sybil 攻击防御缺失)。本扩展方案目标如下：

1. 只允许受信任的节点加入网络——未持有有效证书的节点，无法通过任何 Chord API (join、notify、find_successor 等)
2. 使用非对称加密/签名——每个节点持有独立的公私钥对，不共享密钥
3. 你作为 CA——你是唯一的信任根，颁发证书即代表授权
4. Tracker 不是信任根——即使 Tracker 宕机，节点间身份验证完全自治

1.2 可行性结论

方案完全可行，且与 v1.0 的架构高度兼容。以下是关键设计决策的理由：

决策	选项	理由
签名算法	Ed25519（推荐）而非 RSA/ECDSA	Go 标准库 crypto/ed25519 原生支持，无外部依赖；签名/验签速度比 RSA-2048 快 10-100 倍；私钥 32 字节、公钥 32 字节、签名 64 字节，体积极小
node_id 计算方式	保持 SHA1(uri) 不变	更改 node_id 计算方式（如 SHA1(pubkey)）会破坏手指表和环形路由的所有既有逻辑；URI 唯一性已足够防止 ID 冲突
证书格式	自定义轻量 JSON 证书而非 X.509	X.509 在学习项目中引入不必要的复杂度 (ASN.1、DER 编码)；自定义 JSON 证书完全够用，且更易调试
验证范围	所有节点间 API 均需验证，/chord/ping 和 /chord/identity 除外	ping 和 identity 用于存活探测和诊断，要求签名会阻止 ISOLATED 节点的恢复尝试
时钟容忍	±5 分钟时间窗口	匹配 v1.0 的 5 秒 HTTP 超时语义；防止重放攻击同时容忍合理的时钟漂移

1.3 与 v1.0 的关系

本方案作为叠加层，不修改 Chord 核心算法（第四章）和路由逻辑。变更集中在：

- **新增：**密钥与证书管理（第二章、第三章）
- **新增：**请求签名与验证中间件（第四章）
- **修改：**所有 Node API 的请求/响应头（第七章）
- **修改：**NodeInfo Schema 增加 certificate 可选字段（第五章）
- **新增：**Tracker API 的证书验证（第八章）
- **新增：**边缘情况与攻击面（第九章，26 个场景）

身份模型与密钥体系

2.1 角色定义

系统中存在两类角色：

CA (Certificate Authority) = 你

- 持有 `ca_private_key` (Ed25519 私钥, 32 字节, 离线保存)
- 发布 `ca_public_key` (Ed25519 公钥, 32 字节, 写入每个节点的配置)
- 职责：为每个被授权节点颁发证书；维护可选的吊销列表 (CRL)
- CA 私钥绝对不部署到任何节点上

节点 (Node)

- 每个节点独立持有自己的 `node_private_key` (Ed25519 私钥)
- 持有你签发的 `node_certificate` (含 `node_public_key`)
- 持有 `ca_public_key` (用于验证其他节点的证书)

2.2 节点证书格式

节点证书是一个 JSON 对象，包含以下字段：

```
{
  "version": 1,
  "node_id": "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
  "uri": "https://node1.example.com",
  "public_key": "base64url( Ed25519 公钥, 32 字节 )",
  "issued_at": 1748390400,
  "expires_at": 1779926400,
  "signature": "base64url( CA 用 ca_private_key 对上述字段的 Ed25519 签名, 64 字节 )"
}
```

字段	类型	说明
version	integer	证书格式版本号，当前为 1
node_id	string	40 位十六进制，必须等于 SHA1(uri) (规范化后)
uri	string	节点的规范化 HTTPS URI，必须与节点运行时配置的 node.uri 完全一致
public_key	string	base64url 编码的 Ed25519 原始公钥 (32 字节)
issued_at	integer	Unix 时间戳 (秒)，证书颁发时间

字段	类型	说明
expires_at	integer	Unix 时间戳（秒），证书过期时间
signature	string	CA 对上述五个字段的规范化签名（见 §2.3）

说明

有效期建议：1 年（`expires_at = issued_at + 31536000`）。证书即将过期时节点应在日志中告警，并向 CA 申请续签。

2.3 证书签名规范

为避免 JSON 字段顺序不一致导致签名失败，待签名消息必须按固定规则构造：

```
signed_message = "chord-cert-v1" || "\n"
                || "node_id=" || node_id || "\n"
                || "uri=" || uri || "\n"
                || "public_key=" || base64url(public_key) || "\n"
                || "issued_at=" || issued_at || "\n"
                || "expires_at=" || expires_at
```

- 字段之间用 `\n` 分隔
- 前缀 `chord-cert-v1` 防止跨协议签名复用攻击（见 §9.3）
- 所有字段值均为原始字符串，不做额外编码

实现示例（Go）：

```
func certSignedMessage(c Certificate) []byte {
    return []byte(fmt.Sprintf(
        "chord-cert-v1\nnode_id=%s\nuri=%s\npublic_key=%s\nissued_at=%d\nexpires_at=%d",
        c.NodeID, c.URI, base64.RawURLEncoding.EncodeToString(c.PublicKey),
        c.IssuedAt, c.ExpiresAt,
    ))
}
```

2.4 证书验证步骤

节点收到对端的证书时，必须按以下顺序完成全部验证，任何一步失败均拒绝：

1. **格式检查**：`version == 1`；`node_id` 为 40 位十六进制；`public_key` 可解码为 32 字节；`issued_at < expires_at`
2. **一致性检查**：`node_id == SHA1(uri)`（按 v1.0 §2.1 的规范化规则）
3. **CA 签名验证**：用 `ca_public_key` 验证 `signature` 对应 §2.3 规定的 `signed_message`
4. **有效期检查**：`issued_at <= now <= expires_at`（±5 分钟时钟宽容）
5. **吊销检查**：`node_id` 不在本地 CRL 中（见 §6.2）

6. **URI 一致性检查**：证书中的 uri 必须与请求来源的 URI 一致（防止证书转移，见 §9.5）

第三章

CA 操作手册

3.1 CA 密钥对生成（一次性操作）

```
# 生成 CA 密钥对，以 PEM 格式保存
openssl genpkey -algorithm ed25519 -out ca_private_key.pem
openssl pkey -in ca_private_key.pem -pubout -out ca_public_key.pem
```

或用 Go 工具（推荐，保持工具链统一）：

```
caPublic, caPrivate, _ := ed25519.GenerateKey(rand.Reader)
// 将 caPrivate 写入安全存储，caPublic 发布给所有节点
```

安全规定

- ca_private_key 存放在离线机器或密码管理器（如 1Password、Bitwarden）中
- ca_public_key 可以公开，写入每个节点的配置文件（auth.ca_public_key_base64）

3.2 为新节点签发证书

每次创建一个新节点，执行以下操作：

步骤 1：生成节点密钥对

```
openssl genpkey -algorithm ed25519 -out node1_private_key.pem
openssl pkey -in node1_private_key.pem -pubout -out node1_public_key.pem
```

步骤 2：计算 node_id（规范化 URI 后取 SHA-1）

```
echo -n "https://node1.example.com" | shasum
# a94a8fe5ccb19ba61c4c0873d391e987982fbbd3
```

步骤 3：构造并签名证书（Go 签发工具伪代码）

```
cert := Certificate{
    Version: 1,
    NodeID:  "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
    URI:     "https://node1.example.com",
```

```
    PublicKey: nodePublicKey,
    IssuedAt:  time.Now().Unix(),
    ExpiresAt: time.Now().Add(365 * 24 * time.Hour).Unix(),
}
cert.Signature = ed25519.Sign(caPrivateKey, certSignedMessage(cert))
```

步骤 4：交付给节点

- 将 `node1_certificate.json`（完整证书）和 `node1_private_key.pem` 安全传输给节点运营者
- 推荐通过加密通道（如 SOPS、age 加密文件）传输，避免私钥明文暴露

3.3 吊销节点

吊销不需要任何密码学操作，只需：

1. 将该节点的 `node_id` 加入 `revoked_nodes.json`（CRL 文件）
2. 将新 CRL 推送给所有在线节点（可通过 Tracker 的新 API，见 §8.5，或手动更新配置文件重启）
3. 若需重新授权：生成新密钥对，签发新证书（旧 `node_id` 对应的旧证书失效）

第四章

请求签名协议

4.1 签名覆盖范围

每个经过身份验证的 HTTP 请求必须在 Header 中携带以下四个字段：

Header	示例值	说明
X-Chord-Node-ID	a94a8fe5ccb19ba61c4c0873d391e987982fbbd3	发送方的 <code>node_id</code>
X-Chord-Timestamp	1748390400	Unix 时间戳（秒），用于防重放
X-Chord-Nonce	7f3d2a...（16 字节，hex 编码）	随机 nonce，用于防重放
X-Chord-Signature	base64url(...)	见 §4.2

可选字段（首次握手或对端无缓存时携带）：

Header	说明
X-Chord-Certificate	base64url 编码的完整 JSON 证书（见 §4.5）

4.2 签名消息规范

待签名消息按以下格式构造：

```
signed_message = "chord-request-v1" || "\n"
                || method || "\n"
                || path || "\n"
                || node_id || "\n"
                || timestamp || "\n"
                || nonce || "\n"
                || sha256_hex(body)
```

- **method**：HTTP 方法大写（GET、POST、DELETE）
- **path**：URI 路径（不含 query string，不含域名），如 /chord/notify
- **node_id**：发送方的 40 位十六进制 node_id（与 X-Chord-Node-ID 一致）
- **timestamp**：X-Chord-Timestamp 的值（十进制字符串）
- **nonce**：X-Chord-Nonce 的值
- **sha256_hex(body)**：请求体的 SHA-256 十六进制摘要；GET 请求体为空时使用空串的哈希
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855

签名：X-Chord-Signature = base64url(ed25519.Sign(node_private_key, signed_message))

关键设计

- "chord-request-v1" 前缀防止跨协议签名复用（如将证书签名消息伪造为请求签名）
- 包含 path 防止端点混淆攻击（API A 的签名不能用于 API B，见 §9.6）
- 包含 sha256(body) 防止 CDN/代理篡改请求体（见 §9.7）
- 同时包含 timestamp 和 nonce 实现双重防重放（见 §9.1）

4.3 接收方验证流程

接收方节点在处理每个需要鉴权的请求时，按序执行以下步骤：

1. 提取 Header: node_id、timestamp、nonce、signature
→ 任何字段缺失：返回 401 MISSING_AUTH_HEADERS
2. 时间窗口检查：|now - timestamp| <= 300 秒（5 分钟）
→ 超出窗口：返回 401 TIMESTAMP_OUT_OF_WINDOW
3. Nonce 去重检查：nonce 是否在本地的 nonce_cache 中（见 §4.4）
→ 已存在：返回 401 NONCE_REUSED
4. 证书获取：
→ 若 X-Chord-Certificate 存在：解析并验证证书（见 §2.4），成功后加入 cert_cache (key = node_id)
→ 否则：从 cert_cache 查找 node_id 对应的缓存证书
→ cert_cache 未命中：返回 401 CERTIFICATE_REQUIRED

5. 构造 `signed_message` (见 §4.2)
6. 用证书中的 `public_key` 验证 `signature`
→ 验证失败: 返回 `401 INVALID_SIGNATURE`
7. 将 `nonce` 加入 `nonce_cache` (TTL = 10 分钟)
8. 验证通过, 继续处理业务逻辑

4.4 Nonce 缓存

为防止重放攻击, 每个节点在内存中维护一个 `nonce_cache` :

- **结构:** `map[string]time.Time` (`nonce` → 到期时间)
- **TTL:** 10 分钟 (时间窗口 5 分钟 × 2, 确保窗口内的任意 `nonce` 都能被检测)
- **清理:** 后台 `goroutine` 每 5 分钟扫描并删除已过期条目
- **存储上限:** 最多缓存 10,000 条, 超限时拒绝新请求并告警 (防止 `nonce` 洪水攻击, 见 §9.2)

4.5 证书传递策略

为避免每个请求都携带完整证书 (增加带宽开销), 采用按需携带策略:

- **首次握手:** 总是携带 `X-Chord-Certificate`
- **证书缓存有效期:** 不携带证书, 接收方从 `cert_cache` 取用
- **收到 401 CERTIFICATE_REQUIRED:** 重发请求, 附带 `X-Chord-Certificate`
- **证书缓存过期** (缓存 TTL 建议 1 小时): 下次请求自动携带证书

第五章

NodeInfo Schema 变更

v1.0 的 `NodeInfo` Schema 增加一个可选字段:

```
{
  "node_id": "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
  "uri": "https://node1.example.com",
  "status": "ACTIVE",
  "joined_at": "2026-05-27T12:00:00Z",
  "certificate": {
    "version": 1,
    "node_id": "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
    "uri": "https://node1.example.com",
    "public_key": "base64url(...)",
    "issued_at": 1748390400,
  }
}
```



```
    "expires_at": 1779926400,  
    "signature": "base64url(...)"  
  }  
}
```

certificate 字段使用规则：

- 在 POST /chord/join 的请求体中**必须**携带
- 在 POST /chord/notify 的请求体中**必须**携带（因为这是新节点第一次向后继节点自报）
- 在其他 API 的响应体中为**可选**（用于监控面板展示，普通路由响应不携带）
- 接收方收到后验证证书，通过则缓存至 cert_cache

第六章

证书生命周期管理

6.1 证书续签

节点启动时自检证书有效期，距过期不足 30 天时：

1. 打印 WARN 级别日志：certificate expires in X days, renew required
2. 通过 heartbeat 上报给 Tracker（新增 cert_expires_at 字段，见 §8.4）

续签流程：

1. 你（CA）生成新证书（可复用原有公私钥对，仅更新 issued_at 和 expires_at，重新签名）
2. 将新证书文件传输给节点
3. 节点通过配置热重载或重启加载新证书，无需离网（因为 node_id 和公私钥不变）

6.2 吊销列表（CRL）

CRL 是一个简单的 JSON 文件：

```
{  
  "version": 1,  
  "updated_at": 1748390400,  
  "revoked": [  
    {  
      "node_id": "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3",  
      "revoked_at": 1748390400,  
      "reason": "private key compromised"  
    }  
  ],  
}
```

```
"signature": "base64url( CA 对上述字段的签名 )"
}
```

- CRL 本身也由 CA 私钥签名，节点用 `ca_public_key` 验证后才加载，防止 CRL 被伪造（见 §9.11）
- 分发方式：写入节点配置文件（手动），或通过 Tracker 的 `GET /tracker/crl` 接口下载（见 §8.5）
- 节点每次启动和每个维护周期均重新加载并验证 CRL

6.3 CA 私钥泄漏处理

若 CA 私钥泄漏，是最严重的安全事件。处理步骤：

1. 立刻将网络全部节点下线（或停止接受新节点加入）
2. 生成新的 CA 密钥对
3. 为所有受信任节点重新签发证书（使用新 CA 私钥）
4. 将新 `ca_public_key` 更新到所有节点配置并重启
5. 旧 CA 签发的证书此后在所有节点上验证失败（因 `ca_public_key` 已变更）

核心安全假设

CA 私钥安全即网络安全。CA 私钥应存放在离线设备上，签发操作需要物理接入。可引入 Certificate Transparency 思想：每次签发操作记录在不可篡改的日志文件中，异常签发可被审计发现。

Node API 变更规范

7.1 需要身份验证的接口

#	API 端点	v2.0 验证要求
1	GET /chord/identity	不需要（存活探测，ISOLATED 状态也必须响应）
2	GET /chord/state	需要（包含完整路由表，不对未知节点开放）
3	GET /chord/ping	不需要（存活探测）
4	POST /chord/find_successor	需要
5	GET /chord/predecessor	需要
6	POST /chord/notify	需要，且请求体必须包含 <code>certificate</code>
7	GET /chord/successor_list	需要
8	POST /chord/join	需要，且请求体必须包含 <code>certificate</code>

#	API 端点	v2.0 验证要求
9	POST /chord/leave	需要
10	GET /chord/finger_table	需要

7.2 新增认证相关 HTTP Header

所有需要验证的请求必须携带：

```
X-Chord-Node-ID:      a94a8fe5ccb19ba61c4c0873d391e987982fbbd3
X-Chord-Timestamp:    1748390400
X-Chord-Nonce:        7f3d2a1b9e4c5d6f7a8b9c0d1e2f3a4b
X-Chord-Signature:    base64url(...)
```

视情况携带（首次握手或缓存失效时）：

```
X-Chord-Certificate: base64url( JSON 证书的 UTF-8 字节 )
```

7.3 新增错误码

在 v1.0 §7.2 的错误码基础上新增：

HTTP 状态码	error.code	含义	触发场景
401	MISSING_AUTH_HEADERS	缺少必要的认证 Header	未携带 X-Chord-Signature 等字段
401	TIMESTAMP_OUT_OF_WINDOW	时间戳超出 ±5 分钟窗口	时钟不同步或延迟重放
401	NONCE_REUSED	Nonce 重复使用	重放攻击或客户端 bug
401	CERTIFICATE_REQUIRED	需要但未提供证书	cert_cache 未命中，需携带证书重试
401	INVALID_CERTIFICATE	证书验证失败	CA 签名验证失败、证书已过期、URI 不匹配等
401	CERTIFICATE_REVOKED	证书已被吊销	node_id 在 CRL 中
401	INVALID_SIGNATURE	请求签名验证失败	签名内容不匹配或私钥错误

7.4 POST /chord/join 变更

请求体增加 certificate 字段：

```
{
  "node": {
    "node_id": "f04a8fe5ccb19ba61c4c0873d391e987982fbbd3",
```

```
    "uri": "https://node5.example.com",
    "certificate": { "...": "完整证书对象" }
  }
}
```

接收方逻辑新增步骤（在 v1.0 的 join 处理逻辑之前）：

1. 验证证书（§2.4 全部步骤）
2. 检查请求签名（§4.3 全部步骤）
3. 通过后，继续原有的 find_successor(N.id) 逻辑

7.5 POST /chord/notify 变更

请求体的 node 对象必须包含 certificate：

```
{
  "node": {
    "node_id": "f04a...",
    "uri": "https://node5.example.com",
    "certificate": { "...": "完整证书对象" }
  }
}
```

接收方在处理前验证证书和签名，通过后才执行前驱节点更新逻辑。

第八章

Tracker API 变更规范

8.1 Tracker 的身份验证角色

Tracker 不是信任根，但应执行与节点相同的证书验证，防止未授权节点注册到 Tracker 并被分发给其他节点作为 seed。Tracker 的验证能力有限（它是 Cloudflare Worker，可用 Web Crypto API 执行 Ed25519 验证），且 Tracker 被绕过不影响节点间的安全（节点自己会验证对端证书），但能阻止未授权节点污染 seed 列表。

8.2 POST /tracker/nodes 变更

请求体增加 certificate 字段：

```
{
  "node_id": "a94a8fe5ccb19ba61c4c0873d391e987982fbbd3",
  "uri": "https://node1.example.com",
  "certificate": { "...": "完整证书对象" }
```

```
"certificate": { "...": "完整证书对象" }
}
```

Tracker 验证:

1. 证书 CA 签名是否有效 (使用 `ca_public_key` 配置)
2. 证书 `node_id` 与 `uri` 一致性
3. 证书是否在有效期内
4. `node_id` 是否在 CRL 中

验证通过后才记录节点。

8.3 POST /tracker/nodes/{node_id}/heartbeat 变更

请求体增加 `cert_expires_at` 字段:

```
{
  "status": "ACTIVE",
  "successor_id": "...",
  "predecessor_id": "...",
  "successor_list_size": 3,
  "finger_table_coverage": 0.85,
  "uptime_seconds": 3600,
  "maintenance_cycles": 60,
  "cert_expires_at": 1779926400
}
```

Tracker 在 GET `/tracker/stats` 响应中新增 `expiring_cert_nodes` 字段 (距过期 ≤ 30 天的节点数)。

8.4 GET /tracker/nodes/seeds 变更

返回的 seed 节点列表中每个节点可选地包含 `certificate` 字段, 便于请求方预热 `cert_cache`:

```
{
  "seeds": [
    {
      "node_id": "...",
      "uri": "https://node2.example.com",
      "certificate": { "...": "完整证书对象 (可选)" }
    }
  ],
  "total_known": 15
}
```

8.5 新增 GET /tracker/crl

新增接口, 供节点定期下载 CRL:

请求：GET /tracker/crl

响应 (200)：

```
{
  "version": 1,
  "updated_at": 1748390400,
  "revoked": [
    { "node_id": "...", "revoked_at": 1748390400, "reason": "private key compromised" }
  ],
  "signature": "base64url(...)"
}
```

节点每次维护周期（report_to_tracker 任务中）检查 CRL 是否有更新（可通过 updated_at 对比本地缓存），有更新则下载、验证并加载。

第九章

边缘情况与攻击面分析（26 个场景）

以下 26 个场景覆盖了本方案所有已知的安全边界和边缘情况。

§9.1 重放攻击 (Replay Attack)

场景：攻击者截获一个合法的已签名请求，稍后重新发送该请求，试图触发相同操作（如重复执行 notify 更改邻居关系）。

防御：

- **timestamp 检查：**请求时间戳必须在 ± 5 分钟内，超时即失效
- **nonce 去重：**每个 nonce 只能使用一次，缓存 10 分钟（覆盖完整时间窗口）
- **两者结合：**即使攻击者在 5 分钟内重放，nonce 已在缓存中，同样被拒绝

局限：nonce 缓存仅在内存中，节点重启后缓存清空。若攻击者在节点重启的 5 分钟内重放旧请求，理论上可能成功。缓解：节点重启后在首次维护周期前等待 5 分钟（boot_grace_period = 300s），或将 nonce 持久化到磁盘（复杂度较高，不建议学习项目实现）。

§9.2 Nonce 洪水攻击 (Nonce Flooding)

场景：攻击者发送大量携带不同 nonce 的请求，耗尽节点的 nonce_cache 内存。

防御：

- nonce_cache 设置上限（10,000 条）
- 超过上限时拒绝所有新请求（返回 503 AUTH_OVERLOADED），并记录告警日志
- 配合请求级速率限制（每个 IP 每秒最多 100 次，超出直接 TCP 丢弃）

注意：速率限制需在 TLS 握手之前或之后实现。由于节点使用 HTTPS，推荐在 HTTP 中间件层实现，基于 X-Chord-Node-ID 而非 IP（因为 CDN 后面的 IP 可能是固定的 CDN 出口 IP）。

§9.3 跨协议签名复用攻击（Cross-Protocol Signature Reuse）

场景：攻击者将节点证书的 CA 签名（或其他上下文的签名）拼接到请求中，伪造请求签名。

防御：

- 证书签名消息以 "chord-cert-v1\n" 为前缀
- 请求签名消息以 "chord-request-v1\n" 为前缀
- 两种签名消息结构不同，内容无法互换

§9.4 中间人攻击（MITM）

场景：CDN 或网络中间人拦截并篡改请求，修改请求体后转发。

防御：

- 请求签名覆盖 sha256(body)，篡改 body 后签名失效
- TLS（即使是 CDN 的证书）提供传输层加密，阻止普通窃听
- 两层防御叠加：TLS 防止内容泄露，应用层签名防止内容篡改

局限：若 CDN 拥有节点的 TLS 私钥，它可以解密并篡改内容（但由于它没有节点的 Ed25519 私钥，无法重新签名，篡改后签名验证仍然失败）。

§9.5 证书转移攻击（Certificate Transfer Attack）

场景：节点 A 将自己的有效证书发给未授权节点 B，B 持有 A 的证书但没有 A 的私钥；或攻击者试图将 A 的证书用于不同 URI 的节点。

防御：

- 证书中的 uri 字段与请求来源 URI 绑定，验证时检查一致性（§2.4 步骤 6）
- 请求签名用 node_private_key 生成；B 没有 A 的私钥，无法生成有效签名
- 即使 B 截获了 A 的签名请求，nonce 去重会阻止重放

§9.6 端点混淆攻击（Endpoint Confusion Attack）

场景：攻击者截获对 POST /chord/ping 的合法签名请求，将其中的 path 替换为 POST /chord/join，试图触发 join 操作。

防御：

- 签名消息包含 path 字段，修改 path 后签名失败
- Ping 和 identity 不需要签名，不存在此攻击向量

§9.7 Body 篡改攻击（Body Tampering）

场景：攻击者截获合法的 POST /chord/notify 请求，将 body 中的 node.uri 替换为攻击者控制的 URI，试图污染目标节点的前驱节点信息。

防御：签名消息包含 sha256(body)，篡改 body 后签名失败。

§9.8 Sybil 攻击 (Sybil Attack)

场景：攻击者注册大量恶意节点，在环上占据大量位置，控制特定键的路由，导致路由查询被劫持到攻击者控制的节点。

防御：

- 每个节点必须持有 CA 签发的证书，CA（你）控制颁发数量
- 攻击者无法自行签发证书（没有 CA 私钥）
- 因此，本方案在你控制的私有网络范围内**完全消除**了 Sybil 攻击

§9.9 路由污染攻击 (Routing Poisoning)

场景：攻击者控制一个合法节点（持有有效证书），在 find_successor 响应中故意返回错误的 next_hop，导致路由查询被引导到错误节点。

防御：本方案无法完全防止持有有效证书的节点的恶意行为（这是拜占庭容错问题，超出 Chord 协议的设计范围）。缓解：若某节点频繁导致路由失败，suspicious/failed 机制会将其踢出手指表。根本防御需要拜占庭容错路由（PBFT-Chord 等），远超本学习项目范围。

§9.10 CA 私钥泄漏 (CA Key Compromise)

处理流程见 §6.3。

核心问题：CA 私钥泄漏后，攻击者可以签发任意节点证书，完全接管网络。这是本方案的核心假设——CA 私钥安全即网络安全。

额外缓解：CA 私钥应存放在离线设备上，签发操作需要物理接入。可引入 Certificate Transparency 思想：每次签发操作记录在不可篡改的日志文件中，异常签发可被审计发现。

§9.11 CRL 伪造攻击 (CRL Spoofing)

场景：攻击者伪造 CRL，将合法节点加入吊销列表导致其被网络排斥；或从 CRL 中删除已被吊销节点使其重新被接受。

防御：CRL 由 CA 签名，节点验证 CRL 签名后才加载。攻击者没有 CA 私钥，无法伪造有效的 CRL。

§9.12 时钟偏斜攻击 (Clock Skew Attack)

场景：攻击者通过 NTP 欺骗控制节点系统时间，使时间窗口检查失效，进而实施重放攻击。

防御：nonce 去重机制独立于时钟，即使时间窗口检查被绕过，nonce 重复仍能阻止重放。建议节点使用 NTP over TLS（或 NTS 协议）防止 NTP 欺骗。

§9.13 证书解析攻击 (Certificate Parsing Attack)

场景：攻击者构造畸形 JSON 证书（超长字段、嵌套 JSON 注入、Unicode 混淆），触发解析器崩溃或绕过验证逻辑。

防御：

- 解析 X-Chord-Certificate 时限制 base64 解码后的最大字节数（建议 4KB 上限）
- 解析 JSON 后，不应将字段值传递给任何 shell 命令或数据库查询（本项目为纯内存操作，风险较低）
- node_id 字段必须严格校验为 40 位十六进制，其他格式直接拒绝

§9.14 并发 join 的竞态与证书验证

场景：多个节点同时向同一个 seed 节点发送 POST /chord/join，证书验证和 find_successor 的并发执行可能导致状态不一致。

分析：这不是新问题——v1.0 已有并发 join 分析 (§6.1)。证书验证是无状态的（只读操作），不与 Chord 状态机共享锁，因此不会加剧竞态问题。

§9.15 ISOLATED 节点的认证行为

场景：一个处于 ISOLATED 状态的节点尝试通过 notify 重新连接，目标节点是否应该接受来自 ISOLATED 节点的 notify？

结论：应接受。ISOLATED 节点依然持有有效证书，身份是合法的。ISOLATED 状态是暂时的拓扑问题，不应影响身份认证结果。收到合法 notify 后，接收方按 v1.0 §4.5 的逻辑处理前驱关系。

§9.16 证书缓存中毒 (cert_cache Poisoning)

场景：攻击者先用合法证书建立信任 (cert_cache 中存入合法 public_key)，然后用另一个证书（相同 node_id，不同 public_key）替换缓存，使后续签名验证使用错误的公钥。

防御：

- cert_cache 以 node_id 为 key；相同 node_id 的新证书不覆盖已缓存的证书（“首次信任”原则，Trust-On-First-Use）
- 强制覆盖只发生在：新证书的 issued_at 严格晚于缓存证书的 issued_at，且新证书 CA 签名有效
- 此设计防止攻击者用旧证书覆盖新缓存的证书

§9.17 节点私钥泄漏 (Node Key Compromise)

场景：某节点的私钥文件被盗，攻击者可以伪造该节点的签名。

处理：

1. 吊销该 node_id（加入 CRL）
2. 原节点执行 Graceful Leave
3. 如需重新上线，生成新密钥对，申请新证书（若使用新 URI，node_id 自动更新；若使用原 URI，新证书绑定新公钥，旧公钥在 CRL 的包底下失效）

§9.18 Tracker 返回被吊销节点作为 seed

场景：Tracker 的 CRL 更新滞后，向新节点推荐一个已被吊销的节点作为 seed。

分析：新节点从 Tracker 收到 seed 列表后，若包含 certificate 字段，应立即验证；若 seed 证书已被吊销，跳过该 seed，尝试下一个。

§9.19 手指表条目的证书验证

场景：手指表中存储的是 NodeInfo (node_id + uri)，没有存储证书。当节点通过 fix_fingers 向手指表中的节点发送 find_successor 请求时，对端是否需要再次提供证书？

结论：是。对端节点必须在请求的 Header 中携带认证信息（签名），接收方从 cert_cache 验证。如果 cert_cache 未命中，返回 401 CERTIFICATE_REQUIRED，发起方重发携带证书的请求。这确保了手指表中任何节点请求都经过验证。

§9.20 单节点环的认证

场景：网络中只有一个节点时，它向自身发送 stabilize 的 notify，是否需要签名？

结论：单节点对自身的 notify 是本地函数调用 (v1.0 §6.5 的设计)，不经过 HTTP，因此不涉及签名。若实现中确实发出了 HTTP 请求给自身，则用自己的私钥签名即可——验证时使用自己缓存的自身证书。

§9.21 check_predecessor 的认证

GET /chord/ping 不需要签名 (§7.1)，因此 check_predecessor 的逻辑完全不受影响。

§9.22 Graceful Leave 期间的认证

节点处于 LEAVING 状态时：

- 向 successor 发送 POST /chord/leave：用自己的私钥签名（正常流程）
- 向 predecessor 发送 POST /chord/leave：同上
- 收到来自其他节点的请求：返回 503 NODE_LEAVING（在身份验证中间件执行之后返回，确保未授权节点无法探测 LEAVING 状态）

§9.23 证书有效期与 Chord 网络状态的关系

场景：一个 ACTIVE 节点的证书在运行期间过期，其他节点的 cert_cache 中存储了该节点的过期证书。

处理：

- cert_cache 中的证书在加入时记录其 expires_at
- 若 cert_cache 中的证书已过期 (now > expires_at + 宽容期)，将其从缓存中删除
- 下次向该节点发送请求时，目标节点需要提供新的有效证书（若证书已在 CA 处续签）
- 若证书已过期且未续签：目标节点的所有请求均返回 401 INVALID_CERTIFICATE，最终被标记为 failed 并从手指表中移除

§9.24 DisallowUnknownFields 与认证字段

v1.0 的 README 提到请求体解析使用 DisallowUnknownFields（未知字段被拒绝）。新增的 certificate 字段必须加入 NodeInfo 的 Schema 定义，否则会被拒绝。

结论：更新所有接受 NodeInfo 的 API 的请求体 Schema，将 certificate 字段加为合法字段（可选）。

§9.25 CDN 终止 TLS 后的 URI 一致性

v1.0 使用 HTTPS，CDN（如 Cloudflare）在边缘终止 TLS，然后以 HTTP 或 HTTPS 转发请求至原始节点。此时节点收到的请求来源 IP 是 CDN 的 IP，而非真实客户端。

对认证的影响：

- URI 一致性检查（§2.4 步骤 6）基于 X-Chord-Node-ID 映射到的 uri，而非 IP 地址——CDN 场景下 URI 不变，检查有效
- 请求签名中的 path 是 URL 路径（不含 Host），CDN 不改变路径——签名有效
- 唯一需要注意的：若 CDN 修改了请求体（如添加 CDN 特有字段），sha256(body) 验证会失败。需确保 CDN 透传 /chord/* 路径的请求体不做任何修改

§9.26 配置文件中的私钥安全

节点启动时从配置文件加载 node_private_key，若配置文件权限不当（如 world-readable），私钥可能泄漏。

建议：

- node_private_key 文件权限设置为 0600（仅 owner 可读）
- 支持通过环境变量传入（NODE_PRIVATE_KEY_BASE64），便于容器部署中的 Secret 管理
- 节点启动时检查配置文件权限，若不安全则打印 WARN 日志

新增配置参数

在 v1.0 §9 的基础上，新增以下配置参数：

参数名	默认值	类型	说明
auth.enabled	true	bool	是否启用身份验证。false 时完全回退到 v1.0 行为（用于调试或兼容测试）
auth.ca_public_key_base64	(必填)	string	CA 公钥的 base64url 编码，用于验证节点证书
auth.node_certificate_file	(必填)	string	节点证书 JSON 文件路径

参数名	默认值	类型	说明
auth.node_private_key_file	(必填)	string	节点 Ed25519 私钥文件路径（PEM 或 raw base64）
auth.timestamp_tolerance_seconds	300	integer	时间戳容忍窗口（±秒），默认 ±5 分钟
auth.nonce_cache_ttl_seconds	600	integer	Nonce 缓存 TTL，应为时间戳容忍窗口的 2 倍
auth.nonce_cache_max_size	10000	integer	Nonce 缓存最大条目数，超限后拒绝新请求
auth.cert_cache_ttl_seconds	3600	integer	cert_cache 中已验证证书的缓存时长（秒）
auth.crl_file	" "	string	本地 CRL JSON 文件路径，空则不加载本地 CRL
auth.crl_refresh_from_tracker	true	bool	是否在维护周期内从 Tracker 的 /tracker/crl 自动刷新 CRL
auth.cert_expiry_warn_days	30	integer	证书距过期不足多少天时打印 WARN 日志
auth.boot_grace_period_seconds	0	integer	节点重启后等待多少秒才开始接受请求，设置为 300 可完整防御重启窗口内的重放

消息时序变更（新节点加入）

在 v1.0 §8.1 的时序基础上，步骤 5 和 9 发生变化：

步骤	发起方 → 接收方	v2.0 变更
5	N → S1	POST /chord/join，请求体包含 certificate；Header 携带 X-Chord-* 签名字段
5a	S1 内部	新增： 验证 N 的证书（CA 签名、有效期、URI 一致性）；验证 N 的请求签名；通过后继续 find_successor(N.id)
9	N → A	POST /chord/notify，请求体包含 certificate；Header 携带签名
9a	A 内部	新增： 同步骤 5a，验证证书和签名；通过后处理前驱更新逻辑

实现建议（Go 语言）

12.1 新增内部模块

在 v1.0 的模块基础上，新增：

internal/auth/	# 身份验证核心逻辑
cert.go	# 证书结构体、解析、验证、签名
signer.go	# 请求签名生成（节点发送请求时调用）
verifier.go	# 请求签名验证（HTTP 中间件）
nonce_cache.go	# Nonce 去重缓存
cert_cache.go	# 已验证证书缓存
crl.go	# 吊销列表加载与检查
tools/ca/	# CA 操作工具（仅在你的机器上使用）
main.go	# 生成 CA 密钥对、签发节点证书

12.2 HTTP 中间件集成

在 internal/httpapi 中，将验证逻辑封装为 HTTP middleware：

```
func AuthMiddleware(verifier *auth.Verifier, exempted []string) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            // 豁免端点 (ping, identity) 直接放行
            for _, path := range exempted {
                if r.URL.Path == path { next.ServeHTTP(w, r); return }
            }
            if err := verifier.Verify(r); err != nil {
                writeAuthError(w, err); return
            }
            next.ServeHTTP(w, r)
        })
    }
}
```

豁免路径： /chord/ping、/chord/identity

12.3 无外部依赖原则

v1.0 README 明确 "no external dependencies"。Ed25519 的签名/验签使用 Go 标准库 crypto/ed25519（Go 1.13+），base64url 使用 encoding/base64，SHA-256 使用 crypto/sha256，全部无需任何第三方包，完全符合 v1.0 的设计约束。

版本信息总结

Chord DHT 身份验证扩展设计文档 v2.0 版本信息	
文档版本	v2.0
设计日期	2026 年 5 月 28 日
基于	v1.0 Chord DHT 学习系统设计文档
核心新增	- Ed25519 节点证书（自定义 JSON 轻量格式） - CA 签发流程（离线 CA 密钥对 + 签发工具） - HTTP 请求签名（双重防重放：timestamp + nonce） 26 个边缘情况与攻击面分析（第九章）
Go 依赖变更	无——全部使用标准库 crypto/ed25519、crypto/sha256、encoding/base64，无第三方依赖
node_id 计算	不变，保持 SHA1(uri)（规范化后）
信任根	CA 私钥——你持有，离线保存，绝不部署至任何节点